FACILITATOR STUDY MATERIAL

Day 5 — Agentic AI, Skills, Guardrails and Final Capstone

In-depth explanation of every concept, with analogies, worked examples and delivery guidance

Written so that you can read it once and teach from it confidently across a full eight-hour day. Each concept has: what it means in plain terms, why it matters, the technical detail behind it, an analogy for the room, and the misunderstandings participants typically arrive with.

Enterprise AI Engineering Bootcamp · Companion to the 42-slide Day 5 presentation

How to Use This Document

Read end to end once before Day 5. Sections map to slide numbers in order. Four kinds of box appear throughout: **Analogy**, **How to say it**, **Watch for** and **Deeper detail**.

The two jobs of the final day

First, deliver the agentic and governance material properly — it is genuinely new content and the part most teams have the weakest grip on. Second, get every team to a capstone they can present to their own management on Monday. The second job is the one that determines whether the week produces lasting change, so protect the capstone time even if it means compressing Module 5.

The framing that reduces panic and improves output: **the capstone is assembly, not new work**. Sixteen of the eighteen items already exist in the daily assignments. Say this early and repeat it.

Callbacks to land today

Planted on	Paid off today
Day 2 — 90% to 53% compounding arithmetic	Module 1.2, expanded into a full table and made the design constraint
Day 2 and Day 3 — prompt injection introduced architecturally	Module 4.1 and Lab 4, the hands-on attack
Day 2 — "treat model output as untrusted input"	Module 4.3, output validation and format abuse
Day 3 — confused deputy problem	Module 2.5, reappearing as authorisation laundering between agents
Day 2 — "the Skill registry: Day 5 covers this in full"	Module 3, in full
Day 1 — ablation discipline	Module 5.2, applied to research claims

Timing guide

Slot	Content	Pacing notes
09:00 – 10:15	Module 1 — agentic architecture	Section 1.2 is the centre. Let the table sit before moving on
10:15 – 11:00	Lab 1 — field-service agents	The spare parts agent is the teaching moment
11:00 – 12:15	Module 2 and Lab 2 — agent-to-agent	Frame as interface design; familiar territory for this audience
12:15 – 13:00	Module 3 — the Skill concept	The four-way comparison table in 3.1 is worth walking slowly
13:45 – 14:30	Lab 3 — design a Skill	Insist on the adversarial test case; it primes Module 4
14:30 – 16:15	Module 4 and Lab 4 — guardrails	**The lab teams remember.** Protect the full 45 minutes
16:15 – 16:45	Module 5 — research translation	Deliberately brisk. Compress here if anything has overrun
16:45 – 17:30	Module 6 — capstone preparation	Circulate constantly; most teams need a nudge on the roadmap
17:30 – 18:30	Presentations and close	Twelve minutes each, strictly

Module 1 — Agentic AI Architecture

1.1 What actually makes a system agentic

Not the presence of a large model, and not the ability to call tools. Three properties together: it decides the sequence at runtime; it selects its own tools; and it evaluates its own progress against a goal.

> **The test that settles most arguments.** "Can you draw the flow chart? If you can — even a large one with many branches — you have a workflow. Building it as a workflow gives you the same outcome with a fraction of the cost, latency and failure surface, plus the ability to test it. Agency is warranted when the branching genuinely cannot be enumerated, which is far rarer than the industry's vocabulary suggests."

> **Analogy — the service engineer and the apprentice.** An apprentice given a fault-finding flow chart follows it and produces a consistent result. An experienced engineer confronted with a fault nobody has documented reasons through it, choosing what to check next based on what the last check revealed. The first is a workflow and it is the right tool for known faults. The second is agency and it is the right tool for genuinely novel ones. Neither is better; they solve different problems.

1.2 The compounding error problem

Planted on Day 2, expanded here into the design constraint that shapes everything else.

Steps in the chain	At 95% per step	At 90% per step	At 80% per step
1	95%	90%	80%
3	86%	73%	51%
5	77%	59%	33%
8	66%	43%	17%
12	54%	28%	7%

Three consequences for design:

1. **Shorten the chain.** Every step removed or made deterministic is a multiplicative gain, not an additive one.
2. **Verify between steps, not only at the end.** An error caught at step three costs far less than one discovered at step twelve.
3. **Measure per-step success.** An end-to-end number tells you the agent failed without telling you where.

> **How to deliver it.** Put the table up and stop talking for a moment. Then: "This is why today spends its time on guardrails and observability rather than on making agents more autonomous. Autonomy multiplies error. Everything that follows is a response to this table."

1.3 The agentic patterns

Pattern	How it works	Use when	Main risk
Planner-executor	One plans, another executes	The plan benefits from being reviewable	A bad plan executes fully before anyone notices
Tool-calling agent	One loop selecting tools until done	Simple tasks, few well-described tools	Non-terminating loops; unnecessary calls

Workflow agent	Fixed sequence, model reasoning inside each step	**Steps known, each needs judgement**	Frequently right, and rarely chosen
Reflection agent	Produce, critique, revise	Quality matters more than latency or cost	Doubles cost; self-critique can be shallow
Supervisor-worker	Supervisor delegates to specialists	**Genuinely different sub-tasks**	Supervisor becomes a bottleneck
Multi-agent collaboration	Peers exchange without a supervisor	Rare in enterprise settings	Hardest to debug, audit and reason about

> **The guidance to give.** "Workflow agent and supervisor-worker cover the overwhelming majority of legitimate enterprise use. Start there. Peer collaboration is intellectually appealing and operationally painful — you cannot debug it, you cannot audit it, and you cannot explain to anyone why it did what it did."

1.4 Memory and state

State is this task, right now: what has been done, what is pending, what each step returned, the current plan. **Memory** is across tasks and sessions: preferences, past decisions, what failed before.

> **The failure teams hit first.** Carrying the entire conversation as state. It grows past the context window, cost rises every turn, the lost-in-the-middle effect from Day 3 degrades attention to the earliest instructions, and the run becomes impossible to reason about. Keep state as a structured object your code owns, not as a growing prompt string. Summarise history deliberately. Retrieve memory rather than carrying it.

> **Deeper detail — four kinds of memory worth distinguishing.** Conversation history (this session's exchanges), retrieved knowledge (documents fetched for this task), learned preferences (durable facts about this user or asset), and episodic records (what happened on previous runs of this task). Each needs its own retention, access control and refresh policy — and memory is data, so Day 2's data architecture rules apply to all four. Teams that treat "memory" as one undifferentiated store end up with a permanently growing context and no way to expire anything.

1.5 Task decomposition

1. **Decompose to verifiable units.** A step whose success you cannot assess is a step you cannot recover from.
2. **Make deterministic what can be deterministic.** Lookups, calculations, format conversions — write code. Reserve the model for judgement.
3. **Order by information dependency.** Steps that reduce uncertainty run early.
4. **Fail fast on prerequisites.** Check authorisation and existence before running six steps that will be discarded.
5. **Bound the depth.** Maximum steps, maximum retries, maximum cost. An agent without bounds is an unbounded cost.
6. **Keep the plan visible.** Inspectable before execution — this is what makes an approval gate possible.

> **The single most effective reliability intervention.** "Point two. Every deterministic step you remove from the model is a step that cannot fail probabilistically. Go through your agent design and ask of each step: does this need judgement, or does it need code? Most need code."

1.6 Human approval gates

Gate on	Not on	Why

Irreversibility	Model confidence	Confidence is not a safety property and is frequently miscalibrated
Financial threshold	Step count	Cost of being wrong matters, not how much work was done
Safety relevance	Task complexity	A simple action can be the dangerous one
External visibility	Internal complexity	Anything a customer or supplier sees needs a person
Policy-defined actions	Ad hoc judgement	The gated list should be written down and reviewed

> **Design the gate so it is actually usable.** The approver needs the proposed action, the reasoning, the evidence, and a one-click approve or reject. A gate requiring the approver to reconstruct the agent's reasoning from logs will be rubber-stamped within a fortnight — and a rubber-stamped gate is worse than no gate, because it creates the appearance of control while providing none.

1.7 Failure recovery and observability

Recovery: retry with backoff for transient failures; try an alternative approach on repeated failure rather than the same one again; degrade gracefully with gaps stated; escalate with full context; roll back state-changing steps already taken; and always terminate.

> **The metric that matters most and is measured least.** "Per-step success rate. With the arithmetic from 1.2, a single weak step at 70% drags an otherwise sound eight-step chain below 50%. An end-to-end completion figure tells you the agent failed without telling you where. Instrument every step separately from day one — retrofitting per-step observability onto a running agent is considerably harder than building it in."

Module 2 — Agent-to-Agent Communication

2.1 Why it matters

The moment you have more than one agent, the interface between them determines reliability. Six reasons: ownership boundaries between teams, independent evolution, reuse, auditability, failure isolation, and security boundaries.

> **The framing that works with this audience.** "This is interface design. You have all done it. The novelty is that the component on the other side of the interface behaves probabilistically, which means the contract has to be tighter, not looser."

2.2 Tool or agent?

	Tool	Agent
Does	One well-defined thing	Pursues a goal through multiple steps
Decides	Nothing — it executes	What to do next, and when it is finished
Contract	Typed input, typed output	A goal, constraints, and a defined result shape
Failure mode	Returns an error	May loop, drift, or return something plausible and wrong
Testing	Deterministic unit tests	Scenario-based, statistical, variance across runs
Cost	Predictable per call	Variable — depends on the path chosen

> **The practical rule.** "Prefer tools. Every capability you express as a tool rather than an agent removes a source of non-determinism, becomes unit-testable, and has a predictable cost. Most sub-agents in enthusiastic designs are tools with extra steps."

2.3 Discovery and capability description

Six requirements: a registry rather than hard-coded addresses; capability described in outcome terms; explicit limitations; version and compatibility; ownership and support; and status and maturity.

> **The one teams omit.** Explicit limitations — what it cannot do, which assets it covers, which conditions it is unreliable under. An agent that does not state its limits will be used outside them, confidently, by someone who had no way to know.

2.4 Contracts, delegation and shared context

Element	Specifies	Why it matters
Input contract	What the caller must provide, typed and validated	An underspecified input produces a plausible answer to the wrong question
Output contract	Result structure, including uncertainty	Callers cannot handle free text reliably
Allowed tools	Which capabilities it may invoke	Bounds the blast radius of a compromised or confused agent
Failure condition	What it returns when it cannot succeed	Silence and hallucination are the alternatives
Escalation condition	When it hands to a human	Must be stated, not inferred from confidence

Context passed	What the caller shares and deliberately withholds	**Over-sharing is a leakage path; under-sharing causes wrong answers**

> **The row teams overlook.** Passing the full conversation to a sub-agent is convenient, expensive, and frequently means sending it data it has no authorisation to see. Pass what the contract requires and nothing more. This is both a cost problem and a security problem, and it is invisible until someone audits it.

2.5 Multi-agent failure modes

Failure mode	What it looks like	Mitigation
Compounding error	Each agent reasonable; the chain wrong	Shorten, verify between steps, measure per step
Circular delegation	Agents hand work back and forth	Depth limits, delegation tracking, cycle detection
Goal drift	Output no longer addresses the request	Carry the original goal; verify against it at the end
Context loss	Step-one information absent by step six	Structured state passed forward
Silent degradation	A downstream failure is absorbed	Explicit failure contracts callers must handle
Responsibility diffusion	Wrong outcome, no accountable agent	Recorded exchanges; attributable contributions
Authorisation laundering	A low-privilege request reaches a high-privilege agent	Identity propagation; independent authorisation per agent

The last is Day 3's confused deputy problem, reappearing between agents rather than between client and server. Make that connection explicitly — it shows the pattern recurs wherever privilege boundaries are crossed.

Module 3 — The Skill Concept

3.1 What a Skill is

A packaged, versioned, tested capability for a specific task — instructions, schemas, tool bindings, safety rules and test cases together in one governed unit.

	Prompt	Tool	MCP server	Skill
Is	Text instruction	A callable function	A protocol endpoint	A packaged capability
Versioned	Rarely, in practice	Yes, as code	Yes, as a service	Yes, deliberately
Tested	Almost never	Unit tests	Integration tests	Its own regression suite
Reused	By copy-paste	Yes	Yes	Yes, through a registry
Governed	No	As code	As a service	Reviewed before publication

> **The distinction that matters.** "A Skill is the unit of reuse and governance. The others are its components. A Skill can contain prompts, bind to tools, and call an MCP server — what makes it a Skill is that it is packaged, versioned, tested and owned."

> **Analogy — the standard operating procedure.** Your organisation does not rely on each engineer remembering how to carry out a critical maintenance task. There is a written procedure: steps, tools required, safety precautions, checks, and a revision number. It is reviewed before issue and it is the same document everyone uses. A Skill is exactly that, for a task an AI system performs — and the reason it needs the same governance is the same reason the SOP does.

3.2 Skill components

Eight: instructions, trigger conditions, examples, schemas, tool bindings, deterministic code, safety rules, and test cases.

> **The two that get skipped. Deterministic code** — the parts that should never be probabilistic, which is where the reliability comes from per Module 1.5. And **test cases** — a Skill without them is a shared prompt with ambitions. The suite is what makes reuse safe rather than merely convenient.

3.3 When to create a Skill

Create one when: the task recurs across teams, getting it right requires organisational knowledge, consistency matters, it calls tools needing governance, or someone would otherwise reinvent it quarterly.

Do not when: it is used once by one person, the task is simple and stable, requirements are still changing weekly, a tool or workflow would serve better, or nobody will own it.

> **Five worked examples for this audience.** "A troubleshooting Skill encoding your diagnostic sequence for a product family. A RAG retrieval Skill applying your access rules, version filtering and citation requirements consistently. A test-case generation Skill following your conventions. A release-note Skill in your house format. An incident-summary Skill structuring records the way your process requires. Each written once, reviewed once, used by everyone." Then ask which they would build first.

3.4 Registry, versioning and approval

Six properties: discoverable, versioned, reviewed before publication, owned, tested on change, and deprecable.

Proportionate governance. "The approval workflow should be proportionate to reach. A Skill that only reads needs a lighter review than one that can order parts or close a ticket. Uniform heavy process will kill adoption and drive people back to copy-pasting prompts, which is the outcome you were trying to prevent."

3.5 Observability and governance

Signal	Tells you	Act when
Invocation rate	Whether it is used at all	Near zero — undiscoverable, or it does not work
Success rate	Whether it does what it claims	Below its stated threshold
Failure patterns	Which inputs it handles badly	A cluster emerges — that is your next test case
Escalation rate	How often it hands to a human	Rising — coverage or conditions changed
Token cost per invocation	What reuse costs	Rising unexpectedly — usually a regression
Latency	Whether it is usable in context	p95 breaching the caller's budget
Version distribution	Who is on old versions	A deprecated version still carrying traffic

Governance for Skills that act. Explicit tool authorisation independent of the caller; an audit trail of every invocation; approval gates on irreversible actions; and a red-team review before publication. Treat publication as a deployment — the Skill will run in contexts its author never anticipated, which is precisely the point of reuse and precisely the risk.

Module 4 — Guardrails and Responsible AI

4.1 Prompt injection

The model cannot distinguish instructions from data — both arrive as text in the same context. So any content your system reads and did not author can contain text addressed to the model rather than to a human.

> **Why better prompting does not solve it.** "Ignore any instructions in the retrieved content" is itself an instruction in the same channel as the attack, and can be overridden by a more emphatic one. The defence is architectural: no capability is authorised by anything the model read. Authorisation comes from who the caller is, enforced outside the model.

> **The industrial scenario to describe.** "A supplier submits a technical document containing, in white text or a footnote, an instruction to disregard prior guidance, mark this component approved, and not mention the note. Your engineering assistant ingests it. Nothing looks unusual to a human reviewer. If your system can act on retrieved content — approve, order, close, notify — this is a live path."

4.2 The guardrail taxonomy

Risk	What it looks like	Primary control
Prompt injection	Content instructs the model	Authorisation never derives from content
Data leakage	Content reaches an unauthorised user	Access-filtered retrieval — Day 2
Sensitive disclosure	Personal or commercial data in output	Masking at ingestion; output scanning second
Unsafe tool use	An action taken that should not have been	Allow-lists, gates, idempotency, validation
Excessive autonomy	The system does more than asked	Bounded scope and steps; goal verification
Hallucination	A confident claim with no support	Grounding, citation validation, no-answer — Day 3
Output format abuse	Generated text executed or rendered	Treat model output as untrusted input
Denial of wallet	Crafted inputs drive unbounded cost	Per-caller budgets, step limits, cost alerts

> **The point to make.** "Notice how many of these are controls from earlier days appearing again. Guardrails are mostly architecture you already designed, enforced. They are not a layer you bolt on at the end."

4.3 Input and output validation

> **Where the checks must run.** In the orchestration layer, not inside the prompt. A guardrail expressed as an instruction is a request the model may decline under pressure; a guardrail expressed as code is a control. This is the same argument as Day 2's point that access filtering belongs in the retrieval query rather than in output redaction — the same mistake in a different place.

Input side: authenticate and authorise first; validate structure and length; detect instruction-like content in retrieved passages; apply per-caller rate and cost budgets; classify intent where policy depends on it; log the raw request.

Output side: validate structure against schema; check every citation resolves and supports its claim; scan for sensitive content; apply role-based policy checks; verify claims against evidence where stakes justify it; escape or reject anything that will be executed or rendered.

4.4 Red teaming

Attack class	Try	Success looks like
Direct injection	Instructions in the user's own question	The system follows them over its configuration
Indirect injection	Instructions planted in a retrieved document	The system acts on content rather than caller intent
Access probing	Varied phrasings requesting restricted content	Any leakage, including acknowledging existence
Tool coercion	Phrasing to induce an unauthorised tool call	A tool fires that should not have
Boundary testing	Topics genuinely outside the corpus	A confident answer instead of a decline
Cost attacks	Inputs inducing long chains or repeated retrieval	Unbounded cost or latency on one request

Framing for Lab 4. "Every successful attack becomes a permanent safety test case, exactly as production failures became regression cases on Days 1 and 3. Red teaming is not an event; it is a growing suite. And finding holes today is the successful outcome — the alternative is someone else finding them later, unstructured."

Facilitation note. Expect most teams to succeed on boundary testing and access probing within ten minutes. Teams find this uncomfortable. Reassure them explicitly at the start that discovering failures is the point, and make sure the triage step happens — the question is not just "did it work" but "was the control missing, in the wrong place, or expressed as a prompt instruction rather than as code?"

Module 5 — Research-to-Use-Case Translation

5.1 Reading a paper in twenty minutes

1. **Abstract and conclusion first.** What is claimed and how strongly. Two minutes, eliminates most papers.
2. **The results table.** What was measured, on which benchmark, against which baseline. A weak baseline makes the improvement meaningless.
3. **The setup section.** Dataset size, compute, conditions. This is where enterprise applicability is decided.
4. **The limitations.** Read before the method — it frequently answers whether to continue.
5. **The method, only if still relevant.**
6. **What would have to be true for us.** The translation step, and the one nobody teaches.

Most papers fail at step three for industrial use: the benchmark is public, clean and balanced; your data is proprietary, messy and imbalanced. That is not a criticism of the research — it is the gap you have to bridge, and naming it early saves weeks.

5.2 From a claim to an experiment

Five steps: extract the claim, check applicability, form a hypothesis at Day 1's third level, ablate, and decide and record.

> **Why ablation matters here specifically.** "Papers present complete systems, and the reported improvement is usually the sum of several changes. Adopting the whole system tells you nothing about which part mattered — and frequently one component delivers most of the gain at a fraction of the complexity. Use the ablation table format from Day 1."

5.3 The research backlog and decision records

A **research backlog** scored as on Day 1: expected gain × confidence ÷ effort, reviewed monthly, items age out. **Decision records**: half a page per significant technical decision — what was decided, what alternatives were considered, what evidence supported it, and what would change it.

> **The closing argument for the teaching portion of the week.** "This is the same principle as the regression suite on Day 3, the failure taxonomy on Day 1 and the experiment log on Day 4. In every case: write down what you learned, including the negative results, so the organisation accumulates knowledge rather than repeating work. That is the difference between five years of experience and one year of experience repeated five times."

Module 6 — The Capstone

6.1 Eighteen items, sixteen of which already exist

Items 41 to 58 map onto the week: problem statement, baseline and target metric from Day 1; failure taxonomy and hypotheses from Day 1; architecture pattern, target architecture and data architecture from Day 2; pipeline from Day 3 or 4; MCP design from Day 3; Skill design from today; MLOps from Day 4; guardrails from today; observability from Days 3 and 5; feedback loop and monitoring from Day 4; roadmap from Days 1, 3 and 4; and the 30/60/90 plan produced today.

> **Repeat this framing.** "Today is assembly and honest gap-marking, not new work. Teams that believe this is new work panic and produce worse capstones. Where an item does not exist yet, mark it as a gap with a plan — that is a better answer than inventing something."

6.2 The improvement roadmap — the honest answer to the original brief

The client asked for a route to 95%. This is where the week answers that credibly.

Element	What to state
The metric, named	Not "accuracy" — the specific metric that reflects the decision the system drives
The measured baseline	With its confidence interval and the evaluation set it was measured on
The realistic ceiling	Inter-annotator agreement, or the aleatoric limit of the task as defined
The ranked interventions	Each with an expected delta and the evidence behind the estimate
The cumulative projection	Stated as a range, not a point
What remains	The gap to the target and what would be required to close it

> **The example to give them.** "A roadmap that says 'these five interventions take us from 0.61 to an estimated 0.78 to 0.84, the ceiling is 0.88 given current label agreement, and closing the remainder requires redefining two defect classes' is far more valuable — and far more credible — than one that promises 95%."

6.3 The 30, 60 and 90-day plan

- **First 30 days — measure and diagnose.** Evaluation set, failure taxonomy on real production failures, leakage checklist, baseline with intervals. Deliverable: an honest current-state assessment.
- **Days 31–60 — the cheap wins.** Threshold and calibration; chunking and metadata for retrieval, or label consistency for vision; the top-ranked intervention. Deliverable: a measured improvement with an ablation table.
- **Days 61–90 — make it durable.** Observability, regression suite, feedback capture, deployment path with rollback. Deliverable: a system that can be improved by someone other than you.

> **Resist the model work in the first thirty days.** Every week of this programme has shown why measurement comes first. A plan that starts with measurement is the plan that survives contact with the data — and teams will feel pressure from their management to show model progress in month one. Prepare them for that conversation.

6.4 Presentation format and judging

Twelve minutes per team: ten to present, two for questions. Lead with the honest baseline, not the aspiration. One slide on the failure taxonomy ranked by cost. Architecture on one page. The roadmap with ranges. And name the biggest risk.

> **Set the standard explicitly.** "Teams that name an uncomfortable finding are more credible, not less. Judged on diagnostic honesty, architectural soundness, measurement rigour and realism of the plan — not on how good the numbers look."

Closing the Week

1. A single accuracy number is not a specification.
2. Diagnose before improving.
3. Architecture is the decision with the longest half-life.
4. Retrieval quality, not model quality, drives most RAG outcomes.
5. Autonomy multiplies error.
6. Write down what you learned.

Close by asking each team for the one change they will make first on Monday. It converts a week of material into a commitment, and it gives you a concrete follow-up list.

Companion documents: **Day 5 Presentation** (42 slides with speaker notes) and **Day 5 Question Bank** (anticipated participant questions with model answers).